



UNIVERSIDAD POLITÉCNICA DE MADRID
ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA Y
DISEÑO INDUSTRIAL

Grado en Ingeniería Electrónica Industrial y Automática

PROYECTO DE INFORMÁTICA INDUSTRIAL

**DESARROLLO DEL VIDEOJUEGO ARCHON
MEDIANTE PROGRAMACIÓN
ORIENTADA A OBJETOS EN C++**

RANCHO

Álvaro Pérez Ruano

Julián Osiniri Juárez

Miguel Vázquez Aguado

Jaime Ladero Blanco

David San Román González

Madrid, Mayo de 2026



ÍNDICE

1. INTRODUCCIÓN	3
1.1. Contexto	3
1.2. Origen de "Rancho"	4
1.3. Objetivos del proyecto.....	5
1.3.1. Desacople lógico-gráfico	5
1.3.2. Aplicación de jerarquías polimórficas	5
1.3.3. Implementación de mecánicas duales.....	5
1.3.4. Gestión robusta de memoria	5
1.3.5. Metodología de desarrollo colaborativo	6
1.3.6. Requisitos opcionales	6
2. MANUAL DE INSTRUCCIONES	6
2.1. Requisitos del sistema y dependencias	6
2.2. Guía de inicio	6
2.3. Controles.....	6
2.4. Reglas del juego	6
2.4.1. Condiciones de Victoria.....	6
2.4.2. El Tablero.....	7
2.4.3. Desplazamiento de los Animales	7
2.4.4. La Arena	8
2.4.5. Habilidades del Granjero	8
3. ARQUITECTURA DEL SOFTWARE (POO).....	9
3.1. Desacople entre lógica y motor gráfico	9
3.1.1. Limpieza del Modelo.....	9
3.1.2. Encapsulación y const-correctness	9
3.2. Estructura de clases	10

4. LÓGICA DE JUEGO.....	10
4.1. Fase estratégica (Tablero).....	10
4.1.1. Validación de movimientos	10
4.2. Fase de acción (Arena)	13
4.3. Las piezas (Animales)	16
5. METODOLOGÍA Y CONTROL DE VERSIONES.....	18
5.1. Flujo de trabajo (Gitflow).....	18
5.2. Integración y revisión de código	19
5.3. Estructura del repositorio	20
5.4. Guía de estilo y convenciones de código.....	21
5.4.1. Nomenclatura.....	21
5.4.2. Formato y sintaxis	22
5.4.3. Arquitectura y POO	22
6. ELEMENTOS ORIGINALES Y OPCIONALES	23
6.1. Personalización temática	23
6.1.1. Gráficos y animaciones	23
6.1.2. Música y efectos de sonido.....	24
6.2. Funcionalidades extra	24
7. CONCLUSIONES	24

1. INTRODUCCIÓN

1.1. Contexto

Este proyecto se enmarca en la asignatura de Informática Industrial, como trabajo obligatorio a realizar en el tercer curso del Grado en Ingeniería Electrónica Industrial y Automática ofrecido por la Escuela Técnica Superior de Ingeniería y Diseño Industrial (ETSIDI) de la Universidad Politécnica de Madrid.

El proyecto consiste en el diseño y desarrollo en C++ de un videojuego estructurado bajo el paradigma de la Programación Orientada a Objetos (POO). Tomando como referencia las mecánicas fundamentales del clásico *Archon: The Light and the Dark* (1983) y adaptándolas al entorno moderno de Visual Studio 2026 junto a OpenGL (mediante *freeglut*) y la librería *ETSIDilib* para el renderizado y gestión de recursos multimedia de realización propia.

El código del videojuego se ha concebido desde un inicio respetando la normativa del proyecto, garantizando un fuerte desacoplamiento entre el motor gráfico y las estructuras lógicas, aplicando patrones de diseño y gestionando los recursos dinámicos mediante el patrón RAI (Resource Acquisition Is Initialization) para asegurar la robustez y evitar fugas de memoria.



Figura 1. Archon: The Light and the Dark (1983)

1.2. Origen de "Rancho"

Rancho nace como una reinterpretación original del videojuego clásico *Archon*. Tanto es así que, cambiando un par de letras en forma de anagrama, se puede pasar del nombre original al de este proyecto.



Figura 2. Menú principal de *Rancho*.

Archon: The Light and the Dark está ambientado en un universo de fantasía heroica donde criaturas mitológicas, hechiceros y monstruos se enfrentan en una eterna batalla entre las fuerzas de la luz y la oscuridad. En *Rancho*, esta épica confrontación se traslada a un entorno mucho más terrenal, rústico y cómico: una granja.

La solemne dualidad entre el bien y el mal se sustituye por la feroz rivalidad territorial entre los distintos animales de corral. El tablero de casillas representa las parcelas de la granja que cambian de estado, y la arena de combate simula los caóticos encontronazos que ocurren cuando dos especies se disputan la misma parcela.

Esta temática no solo aporta un tono humorístico y desenfadado, sino que justifica a nivel de diseño de gameplay una rica variedad de estadísticas, habilidades y tipos de ataque. Las diferencias anatómicas y de comportamiento de cada animal encajan a la perfección con la necesidad técnica de diseñar una jerarquía polimórfica variada, permitiendo diferenciar, por ejemplo, los ataques de embestida de una cabra con los lanzamientos de huevo de una gallina.

1.3. Objetivos del proyecto

El objetivo principal de este trabajo es asentar y demostrar de forma práctica los conocimientos sobre Programación Orientada a Objetos adquiridos durante el curso, materializándolos en un programa estructurado, eficiente y libre de errores. Para alcanzar este propósito general, se establecen los siguientes objetivos alineados con la normativa del proyecto.

1.3.1. Desacople lógico-gráfico

Garantizar una separación estricta entre la lógica (estructuras de datos, validación de reglas, estados de colisión) y el motor de renderizado gráfico, de modo que los métodos de las clases del juego no dependan directamente de las librerías gráficas.

1.3.2. Aplicación de jerarquías polimórficas

Modelar las entidades del juego mediante una clase base abstracta Animal que delegue el comportamiento específico a sus clases derivadas mediante enlace dinámico y métodos virtuales, maximizando la escalabilidad y reutilización del código.

1.3.3. Implementación de mecánicas duales

Desarrollar y coordinar dos lógicas de juego independientes pero interconectadas: una fase estratégica por turnos apoyada en matrices (Tablero), y una fase de acción en tiempo real (Arena) que incluya algoritmos de detección de colisiones (AABB) y cinemática en un espacio continuo 2D.

1.3.4. Gestión robusta de memoria

Aplicar rigurosamente el patrón RAII en la construcción y destrucción de pantallas, entidades y recursos multimedia, garantizando la ausencia de fugas de memoria durante el ciclo de vida del programa.

1.3.5. Metodología de desarrollo colaborativo

Emplear un sistema de control de versiones mediante el flujo de trabajo Gitflow. Esto incluye la gestión de temas mediante issues, el aislamiento de funcionalidades en ramas feature/, y la integración de código hacia la rama develop a través de Pull Requests, simulando un entorno de producción profesional.

1.3.6. Requisitos opcionales

Incluir algunos opcionales

2. MANUAL DE INSTRUCCIONES

2.1. Requisitos del sistema y dependencias

Instrucciones detalladas para configurar Visual Studio 2026, freeglut y ETSIDilib (también cómo enlazar dependencias no caben en el .zip).

2.2. Guía de inicio

Pasos para compilar el proyecto.

2.3. Controles

División de inputs (WASD y Flechas), uso del menú/HUD.

2.4. Reglas del juego

La dinámica de "Rancho" combina la planificación táctica por turnos con la acción en tiempo real, basándose en un conjunto de reglas estrictas.

2.4.1. Condiciones de Victoria

Para alzarse con la victoria del rancho, un jugador debe cumplir con uno de los siguientes tres objetivos:

- Controlar los cinco "puntos de poder" (parcelas resaltadas de color verde claro) del tablero simultáneamente.
- Eliminar por completo a todos los animales del oponente.
- Dejar al bando rival con una única pieza y que esta se encuentre bajo el efecto de la habilidad "Atrapar".

2.4.2. El Tablero

La partida se desarrolla sobre un tablero de 9x9 casillas que representa el terreno de la granja. Cada bando comienza la partida con 18 piezas, divididas en 8 especies distintas, que se disponen inicialmente en las dos columnas laterales del tablero.

El escenario presenta dos mecánicas fundamentales:

- Parcelas oscilantes: un tercio de las casillas del tablero cambia cíclicamente su estado (representando el día/noche o afinidad territorial). Combatir en una casilla cuyo estado coincida con la afinidad de la facción otorga ventaja táctica.
- Puntos de poder: existen cinco casillas clave (una en el centro del tablero y cuatro en los centros de cada borde). Los animales que ocupan estas posiciones regeneran su salud más rápido y son inmunes a las habilidades especiales del rival.

2.4.3. Desplazamiento de los Animales

Durante su turno, el jugador puede mover una de sus piezas. La movilidad depende de la anatomía y tipo del animal, clasificándose en tres categorías:

- Terrestres: deben rodear los obstáculos, no pudiendo moverse en diagonal ni atravesar casillas ocupadas.
- Voladores: tienen movimiento libre y pueden sortear obstáculos en su trayectoria, aunque no pueden finalizar su desplazamiento en una parcela ya ocupada por un aliado.
- Tractor: capacidad de moverse a cualquier parcela válida del tablero. El granjero puede coger el Tractor para “teletransportarse”.

2.4.4. La Arena

Cuando un animal intenta moverse a una parcela ocupada por un animal rival, no se produce una captura automática como en el ajedrez. En su lugar, el juego pasa a una arena de combate donde ambos jugadores luchan en tiempo real. El resultado del combate está influenciado por la habilidad de los jugadores y por atributos únicos de cada animal:

- Fuerza de impacto y tipo de arma (ataque cuerpo a cuerpo o lanzamiento de proyectiles).
- Velocidad de movimiento y de ataque.
- Tiempo de recarga (intervalo necesario entre ataques consecutivos).
- Salud máxima.

Al finalizar el combate, el animal perdedor es eliminado (pudiendo darse la eliminación mutua) y el ganador reclama la parcela disputada.

2.4.5. Habilidades del Granjero

El Granjero es la pieza central de cada bando (equivalente al hechicero del juego original) y posee un arsenal de cinco habilidades especiales. Cada habilidad es extremadamente poderosa, pero solo puede utilizarse una vez por partida:

- Teletransporte: mueve inmediatamente a un animal aliado a otra parcela válida.
- Curar: restaura la totalidad de la salud de un animal aliado.
- Intercambio: permuta la posición de dos animales seleccionados.
- Revivir: devuelve a la vida a un animal caído en combate, ubicándolo en una casilla adyacente al Granjero.
- Atrapar: lanza un lazo para atrapar a un animal rival en su casilla actual, inhabilitando su capacidad de movimiento durante varios ciclos del tablero.

3. ARQUITECTURA DEL SOFTWARE (POO)

3.1. Desacople entre lógica y motor gráfico

Uno de los pilares fundamentales en el desarrollo de Rancho ha sido la separación entre la actualización de los datos lógicos y su renderizado gráfico. Al principio del proyecto, las clases del juego como Animal, Tablero o Arena tenían métodos propios para dibujarse en pantalla. Esto generaba un fuerte acoplamiento con las bibliotecas gráficas externas, obligando a que la lógica de negocio dependiera de directivas de bajo nivel de OpenGL, freeglut y ETSIDilib.

Para lograr una arquitectura modular y robusta, se procedió a refactorizar el código centralizando la totalidad del procesamiento visual en una única clase especializada: el Renderizador. Este desacople se articula a través de los siguientes principios.

3.1.1. Limpieza del Modelo

Se eliminaron por completo los métodos `dibuja()` y las inclusiones de bibliotecas gráficas en las clases lógicas. Objetos como Animal o Menu pasaron a ser contenedores puros de estado (posiciones, vida, temporizadores), ignorando qué representación visual (sprite o modelo) les corresponde en pantalla. De este modo, la lógica del juego podría ser ejecutada teóricamente en una consola de texto sin requerir modificaciones en su código fuente.

3.1.2. Encapsulación y const-correctness

Para garantizar que el motor gráfico actúe exclusivamente como una "Vista" observadora y no altere accidentalmente la lógica del juego, la comunicación se realiza mediante punteros constantes. Las clases exponen su estado a través de métodos getters etiquetados con la palabra reservada `const`.

Si el Renderizador necesita pintar la el tablero, recibe un objeto inmutable, asegurando la integridad de los datos:

Fragmento 1. Ejemplo.

```
void Renderizador::dibujar(const Tablero* tablero) const {  
    // DIBUJAR FONDO Y LETRERO DE TURNOS  
    dibujarSprite("../assets/Sprites/tablero/tableroFondo.png", 512, 512, 480  
    / 2, 270 / 2, -1);  
  
    for (int i = 0; i < Constantes::FILAS_TABLERO; i++) {  
        for (int j = 0; j < Constantes::COLUMNAS_TABLERO; j++) {  
            const Animal* animal = tablero->getAnimalEnCasilla(i, j);  
            if (animal != nullptr) {  
                this->dibujar(animal);  
            }  
        }  
    }  
}
```

Máquina de Estados y Patrón Comando en la clase coordinadora, Juego.

Gestión de memoria y recursos (RAII). Patrones GRASP.
Regla de los Tres/Cinco/Cero (falta decidir cual, creo que Cero es la mejor).
Principios SOLID.

3.2. Estructura de clases

Abstracción, Encapsulamiento, Herencia y Polimorfismo.

El coordinador (Juego) y la gestión de fases (Menú, Tablero, Batalla).

Jerarquía polimórfica: clase abstracta Animal y sus hijas (Gallina, Cerdo, etc.), usando métodos virtuales puros y enlace dinámico.

Estructuras de control: Tablero (matriz de punteros) y Arena (fase de acción).

4. LÓGICA DE JUEGO

4.1. Fase estratégica (Tablero)

Validación de movimientos, oscilación de casillas y gestión de turnos.

4.1.1. Validación de movimientos

El sistema de movimiento en el tablero se ha diseñado bajo el patrón de separación de responsabilidades, desacoplando estrictamente la representación gráfica interactiva de la

lógica de reglas del juego. Durante su turno, el jugador puede desplazar el cursor libremente y mantener una pieza en el aire, pero el motor central del tablero evalúa el movimiento basándose exclusivamente en dos parámetros discretos: la casilla de origen y la casilla de destino.

Para gestionar esta transferencia de estado, dentro de un archivo *estructuras.h* se han definido estructuras de datos dedicadas que representan las coordenadas lógicas en la matriz del tablero.

Fragmento 2. Estructuras base para la gestión posicional.

```
struct Casilla {  
    int fila;  
    int columna;  
};  
  
struct Movimiento {  
    Casilla origen;  
    Casilla destino;  
};
```

Para maximizar la reutilización de código y evitar la redundancia (Principio DRY), el cálculo del rango de celdas alcanzables se gobierna de forma centralizada en la clase base *Animal*. Dado que la geometría de expansión radial alrededor de la pieza es idéntica para las diferentes piezas del juego, el método *movimientosPosibles()* no requiere ser redefinido individualmente en cada subclase.

En su lugar, el método de la clase madre pregunta de manera dinámica al atributo encapsulado *max_casillas_movidas_*. Este valor es configurado de manera específica en el constructor de cada animal dependiente (por ejemplo, asignando un alcance de 3 para la gallina y de 4 para la oveja). De este modo, la clase base resuelve la cuadrícula teórica de destinos válidos para cualquier pieza del rancho, reduciendo el coste de mantenimiento del software y facilitando la escalabilidad ante la futura incorporación de nuevos animales.

Fragmento 3. Cálculo polimórfico del alcance en la clase Animal.

```
std::vector<Movimiento> Animal::movimientosPosibles() const {  
    std::vector<Movimiento> movimientos;  
    Casilla origen = { casillaInicial_[0], casillaInicial_[1] };  
    int alcance = max_casillas_movidas_;
```

```

        for (int f = -alcance; f <= alcance; f++) {
            for (int c = -alcance; c <= alcance; c++) {
                if (f == 0 && c == 0) continue;

                int nuevaFila = origen.fila + f;
                int nuevaCol = origen.columna + c;

                if (nuevaFila >= 0 && nuevaFila <
Constantes::FILAS_TABLERO &&
                nuevaCol >= 0 && nuevaCol <
Constantes::COLUMNAS_TABLERO) {

                    Movimiento m;
                    m.origen = origen;
                    m.destino = { nuevaFila, nuevaCol };
                    movimientos.push_back(m);

                }
            }
        }
    }
    return movimientos;
}

```

Cuando el usuario interactúa con el teclado para confirmar el posicionamiento de una pieza, la clase Tablero procesa la solicitud mediante el método `esMovimientoLegal(const Movimiento& m)`. El algoritmo descarta movimientos que incurran en fuego amigo, evaluando que la casilla de destino no contenga un puntero a un animal del mismo equipo, y comprueba si la coordenada seleccionada se encuentra registrada dentro del contenedor de movimientos permitidos generado por el objeto Animal.

Una vez ratificada la legalidad de la jugada, el método `mover(const Movimiento& m)` efectúa la transferencia en la matriz de punteros y actualiza las propiedades físicas de renderizado de la pieza para asentarla en su nueva posición. Durante este proceso, resulta necesario reiniciar los vectores de velocidad residuales del objeto para garantizar que la rutina de actualización gráfica mantenga el sprite perfectamente estático en el centro de la casilla lógica asignada.

Fragmento 4. Sincronización espacial.

```

void Tablero::mover(const Movimiento& m) {
    Animal* pieza = jugadores_[turno_actual_]->getPiezaSeleccionada();

    // 1. Actualización de la matriz lógica
    casillas_[m.destino.fila][m.destino.columna] = pieza;

    // 2. Corrección de simetría vertical y sincronización gráfica
    float nuevaPosX = 141.0f + 11.0f + (22.0f * m.destino.columna);
    float nuevaPosY = 36.0f + 11.0f + (22.0f * (8 - m.destino.fila));
}

```

```

pieza->setPosX(nuevaPosX);
pieza->setPosY(nuevaPosY);

// 3. Freno de la inercia visual
pieza->setVelX(0);
pieza->setVelY(0);
pieza->setEnMovimiento(false);
}

```

4.1.2. Habilidades especiales

El Granjero actúa como la pieza central de cada equipo y tiene un arsenal de habilidades tácticas especiales equivalentes a los hechizos del juego original. Para integrar estas acciones sin romper el flujo estratégico por turnos del tablero ni acoplar en exceso las clases, se ha diseñado e implementado una Máquina de Estados finitos dentro de la clase Tablero. El sistema define un enumerado EstadoHechizo que permite al motor del juego pasar desde un estado inactivo hacia diferentes fases de selección de objetivos (aliados, enemigos o casillas vacías), dependiendo de la habilidad invocada mediante el teclado.

Fragmento 5. Máquina de estados para el control de habilidades.

```

enum EstadoHechizo
{
    INACTIVO,
    TELETRANSPORTE_SELECCIONAR_ALIADO,
    TELETRANSPORTE_SELECCIONAR_DESTINO,
    CURAR_SELECCIONAR_ALIADO,
    INTERCAMBIO_SELECCIONAR_ALIADO,
    INTERCAMBIO_SELECCIONAR_ENEMIGO,
    ATRAPAR_SELECCIONAR_ENEMIGO
};

```

Al activarse una habilidad, el tablero intercepta los comandos lógicos habituales. En lugar de permitir el desplazamiento de la pieza agarrada, el sistema evalúa la interacción del cursor según el estado activo a través del método ejecutarPasoHechizo(). Para asegurar una experiencia de usuario clara y evitar interferencias visuales, el renderizado gráfico de las casillas de movimiento posibles se supedita estrictamente a que la máquina se encuentre en estado INACTIVO.

Respecto al balance táctico del juego, la normativa dice que cada habilidad es extremadamente poderosa y solo puede ser utilizada una vez por partida. Para gestionar

y restringir este recurso limitante, se ha incorporado una estructura de datos matricial que almacena de forma persistente la disponibilidad de las habilidades para ambos bandos:

Fragmento 6. Matriz de disponibilidad de un solo uso.

```
bool hechizoDisponible_[2][5];
```

Esta validación se comprueba al inicio de la rutina procesarTeclaHechizo(). Si el hechizo se completa con éxito (por ejemplo, al sanar un aliado o permutar posiciones), la posición correspondiente en la matriz cambia a false, consumiendo el hechizo.

Finalmente, para las habilidades que introducen efectos de estado prolongados, como es el caso de Atrapar (que inmoviliza a una unidad rival impidiéndole el movimiento), se ha encapsulado la lógica de control temporal en el método avanzarTurnosAtrapados(). Para garantizar un castigo íntegro y evitar un descuento de turnos prematuro, este método se invoca sistemáticamente tras procesar cualquier jugada legal, justo antes de transferir el turno lógico al jugador contrario. Esto asegura la sincronización entre las restricciones de las piezas y el ciclo del tablero.

4.2. Fase de acción (Arena)

Algoritmos de colisión (AABB), confinamiento en los límites de la pantalla y gestión dinámica de proyectiles/ataques.

En esta recreación del Archon se buscó añadir mayor aleatoriedad en la jugabilidad del escenario arena, para ello se tomó la decisión de crear una función que colocaría las barreras de este escenario de juego de manera aleatoria en cada combate.

Para ello se decidió primero, dividir la zona de combate en 8 zonas diferentes, para que cada una de estas zonas esté ocupada por una de las ocho barreras que incluye nuestra arena. Para lograr esto se usaron las constantes ZONA_DE_COMBATE_X e Y, las cuales indican cual es el límite de las zonas de arena, y una variable interna de la función, margen, la cual evita que las barreras se coloquen en los bordes de los límites de la zona de pelea. Si se presta atención a la lógica de la función, se ve, que las variables zona_x y zona_y indican en cuantas filas y columnas se ha dividido la zona de

combate, en este caso se ha dividido la zona de combate en 2 filas (zona_x) y 4 columnas (zona_y).

Una vez ya dividida las zona de la arena en mini zonas, se calcula que mini zona va a ocupar cada barrera, siendo así que, si se sigue la lógica del bucle de la función, y sabiendo que NUM_DE_BARRERAS = 8, la barrera 0 ocupa la columna 0 y fila 0 del área dividida de la arena, la barrera 1 ocupa la columna 1 y fila 0 de la zona dividida, y así sucesivamente.

Una vez ya calculado que barrera va a ocupar cada mini zona, se calculan los puntos de origen de posición de las barreras dentro de las mini zonas, que serán las esquinas de sus mini zonas respectivas, para más tarde determinar cual va a ser la posición final de cada barrera dentro de su mini zona. Para ello se suma al punto de origen de cada barrera un valor aleatorio contenido en su zona de ocupación (multiplicado por 0.7 para evitar que barreras vecinas se posicionen muy juntas). Una vez calculada la posición final, se obtiene que las 8 barreras de arena se posicionan de formas diferentes en cada combate dentro del área de juego de este escenario

Fragmento 7. Ejemplo.

```
void Arena::colocarBarrerasAleatorias()
{
    float margen = 15.0f;
    float zona_x = (ZONA_DE_COMBATE_X - margen * 2) / 4.0f;
    float zona_y = (ZONA_DE_COMBATE_Y - margen * 2) / 2.0f;

    for (int i = 0; i < NUM_DE_BARRERAS; i++) {
        int columna = i % 4;
        int fila = i / 4;

        // esquina de cada zona dentro del cuadrado azul
        float origen_x = ARENA_MARGEN_X + margen + (columna *
zona_x);
        float origen_y = ARENA_MARGEN_Y + margen + (fila * zona_y);

        // posicion aleatoria dentro de esa zona
        barrera_x[i] = origen_x + (rand() % (int)(zona_x * 0.7f));
        barrera_y[i] = origen_y + (rand() % (int)(zona_y * 0.7f));
    }
}
```


4.3. Las piezas (Animales)

Características de cada animal, tipo, vida... Foto del animal.

Para el granjero, habilidades equivalentes a los hechizos del Archon original.

En esta recreación del Archon se optó que en cada equipo de cada bando dispusiera de 9 gallinas, 2 cabras, 2 llamas, 2 cerdos, 2 ovejas y un granjero.

Gallina:



Si se usan términos relacionados con el ajedrez, la gallina dentro de este juego actúa como peón, esta unidad es la que más repite en cada equipo, y es la primera línea de figuras de cada bando.

Si se inspecciona su clase respectiva, es decir, gallina.h, se puede obtener más datos relacionados con esta unidad.

Clase gallina.h:

```
#pragma once
#include "animal.h"
#include "disparo.h"

class Gallina : public Animal
{
public:
    Gallina(Casilla casillaInicial, int equipo) : Animal(casillaInicial,
equipo)
    {
        vida_ = 14;
        max_casillas_movidas_ = 2;
        nFrames = 8;
        especie_ = GALLINA;
        ataque_ = new Disparo(2, 140.0f, 0.8f,
        "../assets/Sprites/menu/selector.png", 15.0f, 0.35f);
    }

    const char* getTipoAtaque() const override { return "Disparo"; }
};
```

Si se presta atención a las líneas incluidas dentro de esta clase, se puede apreciar que la unidad de la gallina dispone de 14 puntos de vida, y que dentro del escenario tablero, esta unidad puede moverse un máximo de 2 casillas, en la siguiente imagen se presenta de forma clara que movimientos dentro de tablero tiene permitido la gallina realizar:



Por otro lado, dentro de esta clase se puede observar también que tipo de ataque dispone esta unidad dentro del escenario arena, el ataque es uno de tipo disparo. Si se accede a la clase `disparo.h`, dentro de su constructor se localizan los siguientes parámetros.

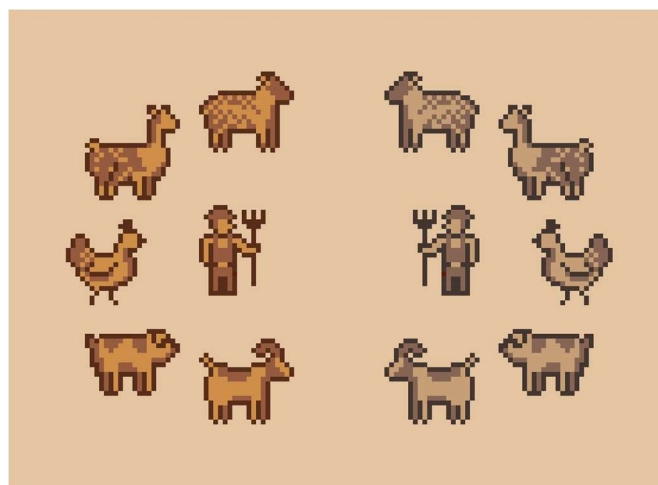
```
Disparo(int dano, float alcance, float recarga, const char* sprite, float
tamano, float duracion)
```

Sabiendo los parámetros del constructor, se puede llegar a la conclusión que el ataque disparo de la gallina posee una capacidad de daño de 2 puntos de vida, un alcance de hasta 140 píxeles de pantalla, con un tiempo de recarga de 0.8 segundos, tamaño 15 píxeles, y una duración de 0.35 segundos.

Cabra:



Por otro lado, la unidad cabra son las piezas localizadas en las esquinas de ambos equipos,



5. METODOLOGÍA Y CONTROL DE VERSIONES

Para garantizar el éxito del proyecto, la colaboración fluida entre los cinco miembros del equipo y la integridad del código se ha establecido una metodología de trabajo rigurosa basada en estándares profesionales de la industria del software.

5.1. Flujo de trabajo (Gitflow)

La gestión del control de versiones se ha articulado en torno a la metodología Gitflow, estructurando el repositorio en diferentes niveles de aislamiento para proteger el código funcional:

- Rama master: contiene únicamente las versiones estables y funcionales del juego (releases). Es el reflejo del producto final.
- Rama develop: actúa como la rama principal de integración. Todo el código nuevo se fusiona aquí para ser testeado en conjunto.
- Ramas feature/: todo nuevo desarrollo se aísla en su propia rama a partir de develop. Para organizar el trabajo en "Rancho", se han utilizado ramas temáticas específicas como feature/animales, feature/tablero, feature/arena, feature/juego, feature/teclado o feature/creditos.
- Ramas hotfix y release: reservadas para correcciones críticas de errores en producción y para la preparación de versiones estables, respectivamente.

Branch	Updated	Check status	Behind	Ahead	Pull request
develop  	 yesterday			Default	
feature/juego 	 19 hours ago		0	9	 #45
feature/clase-jugador 	 yesterday		25	2	
feature/animales 	 2 days ago		19	3	
master  	 2 months ago		78	0	

Figura 3. Ramas del repositorio en GitHub.

5.2. Integración y revisión de código

Para mantener la estabilidad de la rama develop, se ha prohibido la subida directa de código (commits directos). La integración de las ramas feature/ se realiza exclusivamente a través de Pull Requests (PR).



Figura 4. Pull Request para fusionar el menú principal a la rama develop.

Este sistema asegura que cualquier nueva funcionalidad implementada por un miembro del equipo sea revisada y validada antes de fusionarse. El uso de PRs permite la resolución temprana de conflictos de fusión (merge conflicts) y asegura que el código entrante cumple con la guía de estilo del proyecto.

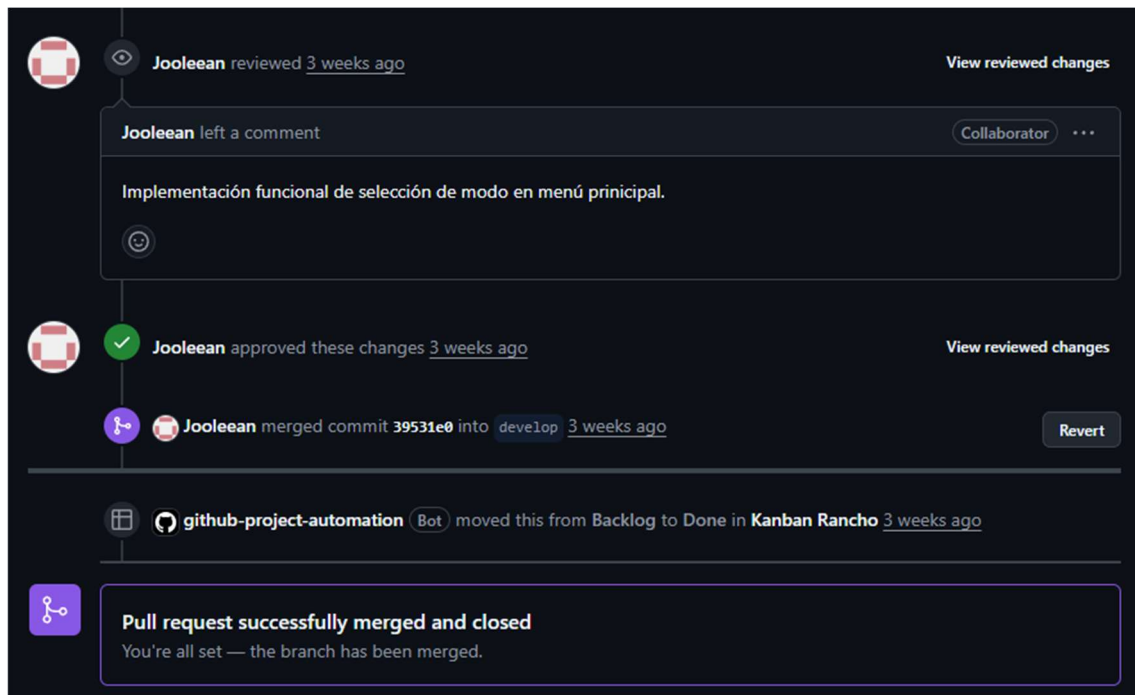
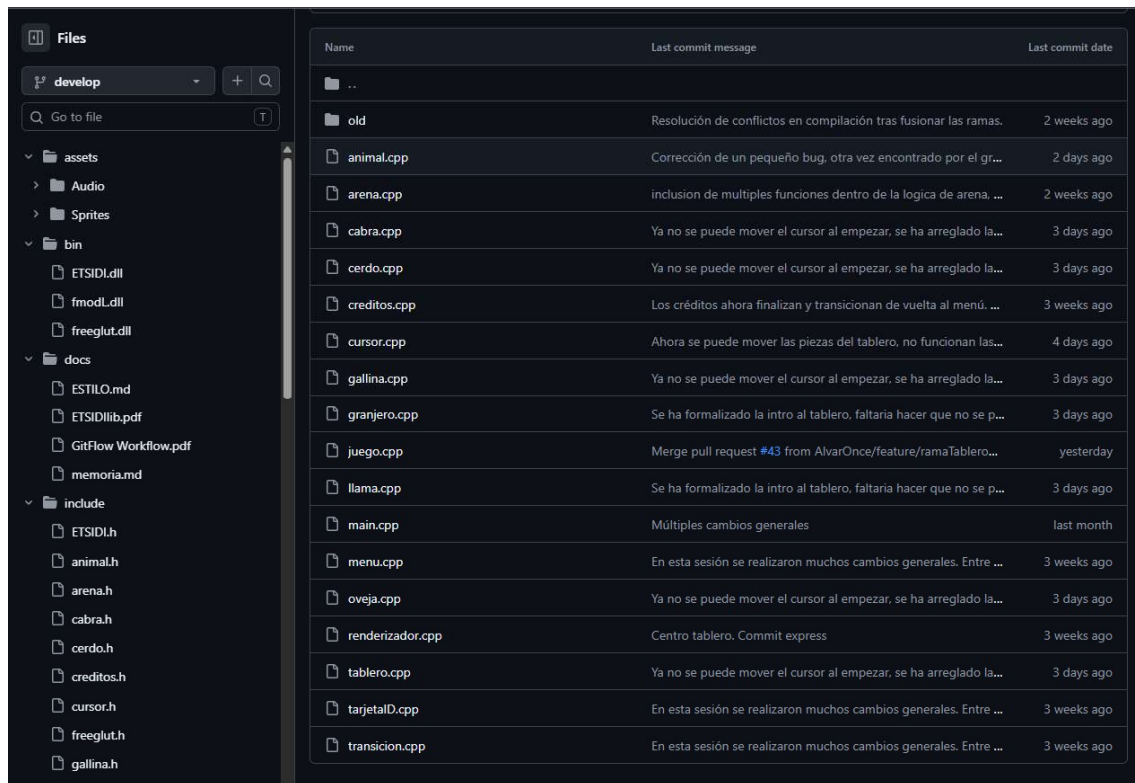


Figura 5. Cambios aprobados tras revisar los archivos cambiados.

5.3. Estructura del repositorio

El repositorio de GitHub se ha organizado siguiendo una estructura de directorios limpia y modular, separando el código fuente de los recursos y la documentación:

- assets/: Almacena los recursos multimedia del juego, subdividido a su vez en Audio (efectos y música) y Sprites (texturas e imágenes de los animales, tablero, etc.).
- bin/: Contiene las librerías dinámicas necesarias para la ejecución del programa en entornos Windows (ETSIDI.dll, freeglut.dll).
- docs/: Recopila la documentación técnica del proyecto, incluyendo manuales (ESTILO.md), instrucciones de librerías y la presente memoria.
- include/: Directorio reservado para los ficheros de cabecera (.h), donde se declaran las clases y estructuras (ej. animal.h, tablero.h, arena.h).
- src/: Contiene los ficheros de código fuente (.cpp) donde se definen e implementan los métodos de las clases (ej. animal.cpp, main.cpp, juego.cpp).



Name	Last commit message	Last commit date
...		
old	Resolución de conflictos en compilación tras fusionar las ramas.	2 weeks ago
animal.cpp	Corrección de un pequeño bug, otra vez encontrado por el gr...	2 days ago
arena.cpp	inclusion de multiples funciones dentro de la logica de arena. ...	2 weeks ago
cabra.cpp	Ya no se puede mover el cursor al empezar, se ha arreglado la...	3 days ago
cerdo.cpp	Ya no se puede mover el cursor al empezar, se ha arreglado la...	3 days ago
creditos.cpp	Los créditos ahora finalizan y transicionan de vuelta al menú. ...	3 weeks ago
cursor.cpp	Ahora se puede mover las piezas del tablero, no funcionan las...	4 days ago
gallina.cpp	Ya no se puede mover el cursor al empezar, se ha arreglado la...	3 days ago
granjero.cpp	Se ha formalizado la intro al tablero, faltaria hacer que no se p...	3 days ago
juego.cpp	Merge pull request #43 from AlvarOnce/feature/ramaTablero...	yesterday
llama.cpp	Se ha formalizado la intro al tablero, faltaria hacer que no se p...	3 days ago
main.cpp	Múltiples cambios generales	last month
menu.cpp	En esta sesión se realizaron muchos cambios generales. Entre ...	3 weeks ago
oveja.cpp	Ya no se puede mover el cursor al empezar, se ha arreglado la...	3 days ago
renderizador.cpp	Centro tablero. Commit express	3 weeks ago
tablero.cpp	Ya no se puede mover el cursor al empezar, se ha arreglado la...	3 days ago
tarjetaID.cpp	En esta sesión se realizaron muchos cambios generales. Entre ...	3 weeks ago
transicion.cpp	En esta sesión se realizaron muchos cambios generales. Entre ...	3 weeks ago

Figura 6. Estructura del repositorio.

5.4. Guía de estilo y convenciones de código

Para garantizar la calidad, legibilidad y estabilidad del proyecto entre todos los desarrolladores, se ha redactado y aplicado una guía de estilo basada en las convenciones Google C++ Style Guide.

Fragmento 8. Ejemplo de estilo y sintaxis.

```
void Animal::animar(float dt) {
    timer = timer + dt;

    if (timer > msStep) {
        if (frameActualX_ < nFrames-1) frameActualX_++;
        else frameActualX_ = 0;
        timer = timer - msStep;
    }
}
```

5.4.1. Nomenclatura

- Clases y estructuras (PascalCase): la primera letra de cada palabra en mayúscula (ej. class Tablero, struct CoordenadaArena).

- Métodos y funciones (`camelCase`): la primera letra en minúscula y las siguientes palabras inician en mayúscula (ej. `void actualizarEstado()`, `bool movimientoValidoTablero()`).
- Variables locales y parámetros (`snake_case`): todo en minúsculas, separando palabras con guiones bajos (ej. `int vida_actual`, `float posicion_x`).
- Atributos de clase (`snake_case_`): los miembros privados terminan con un guion bajo para diferenciarlos rápidamente de las variables locales dentro de los métodos (ej. `Tablero* tablero_juego_`, `int dano_base_`).
- Constantes y enums (`SCREAMING_SNAKE_CASE`): todo en mayúsculas con guiones bajos (ej. `MAX_ANIMALES`, `enum class EstadoJuego { MENU, TABLERO, BATALLA }`).
- Ramas del repositorio: emplean el formato `feature/kebab-case` (ej. `feature/entradas-por-teclado`).

5.4.2. Formato y sintaxis

- Llaves `{ }`: la llave de apertura se coloca en la misma línea que la declaración (métodos, bucles, condicionales). La de cierre va en una nueva línea alineada con la declaración.
- Sangría: se emplean estrictamente 4 espacios por cada nivel de indentación.
- Punteros y Referencias: el asterisco (*) o el ampersand (&) se escriben pegados al tipo de dato, no a la variable (ej. `Animal* gallina`;, no `Animal *gallina`).
- Getters y Setters: el getter debe llamarse igual que el atributo (sin el guion bajo final y sin el prefijo `get`) y el setter debe empezar con el prefijo `set_` (ej. `vida()` y `set_vida()`).

5.4.3. Arquitectura y POO

- Protección de cabeceras: todo archivo `.h` debe comenzar con la directiva `#pragma once` para evitar inclusiones múltiples.
- Encapsulamiento: los atributos de clase son siempre `private` o `protected`, ubicándose por defecto al principio de la clase. Los métodos de acceso se

implementan únicamente cuando es estrictamente necesario exponer o alterar un dato desde el exterior.

- Const-correctness: se etiqueta con la palabra reservada `const` a todos aquellos métodos observadores que consultan datos pero no alteran el estado interno del objeto (ej. `int setVida() const;`).
- Gestión de memoria: siguiendo el principio de asimetría, toda asignación dinámica de memoria realizada con `new` tiene asegurada su correspondiente liberación con `delete` en el destructor de la clase, garantizando el cumplimiento del patrón RAII y evitando fugas de memoria.

6. ELEMENTOS ORIGINALES Y OPCIONALES

6.1. Personalización temática

Elementos únicos de "Rancho" (gráficos, música, sonidos).

6.1.1. Gráficos y animaciones

Fragmento 9. Ejemplo.

```
void dibujarSprite (
    const char* rutaImagen, // ruta del sprite a dibujar
    float ancho, // ancho y alto del sprite, si es un sprite sheet, el ancho y
    alto total del sprite sheet, no del frame individual
    float alto,
    float posx, // posición del centro del sprite en la ventana, no de la
    esquina superior izquierda
    float posy,
    float capa, // la capa en la que se dibuja el sprite, cuanto mayor sea el
    número, más cerca del espectador estará el sprite (capa = profundidad)
    int rows = 1, // número de filas y columnas del sprite sheet, si el sprite
    no tiene animación, se deja en 1
    int cols = 1,
    int stateX = 0, // número de frame en la dirección horizontal (columna),
    empezando por 0
    int stateY = 0,
    bool transparencia = true // si el sprite tiene transparencia, se activa
    el alpha test para no dibujar los píxeles completamente transparentes, esto
    mejora rendimiento y evita artefactos raros entre animales
) const;
```


6.1.2. Música y efectos de sonido

6.2. Funcionalidades extra

Menú principal, créditos, guardar partida, pantalla final con estadísticas de la partida

7. CONCLUSIONES

Retos superados.

Trabajo futuro, posibles mejoras...